

DMS 技术文档

DMS 文档管理系统 —— 技术文档

- > 版本：1.0.0 目标系统：Debian 13 (trixie) / amd64
- > 本文面向接手这套系统的开发与运维人员：讲清它由什么组成、关键设计为什么这么定、数据怎么流。
- > 部署步骤见《03-部署手册.md》，日常运维见《04-运维手册.md》，依赖版本见《02-依赖与版本清单.md》。

1. 系统定位

内网（生产现场）使用的**文档管理系统**，替代共享文件夹式的资料管理：

能力	说明
文档区 / 目录	顶层「文档区」由超级管理员建立，下设任意层子目录
上传下载	浏览器直传（tus 断点续传，中断可续），支持 10GB 单文件；下载分为单文件、目录打包 zip
在线预览	图片、视频、PDF、Office、CAD、压缩包等 75+ 种格式；Office 走 OnlyOffice，其余走服务端转换 + 浏览器原生/pdf.js
权限	「禁止访问 / 只读 / 读写 / 完全控制」四档 + 独立的「下载」开关，可授给用户、角色、部门，支持目录继承与到期时间
审计	上传、下载、打包下载、授权/撤销、删除、重命名、移动等敏感操作全部留痕，可按动作/人/时间段查询与导出
用户自助	站点名称/标识图可配置；可开关自助注册、邮箱验证码找回密码
运维可见	「系统信息」页展示服务器/应用/数据库/对象存储/缓存的实时状态与业务数据量

****规模定位****：单机部署，支撑几十到几百用户、十万级文档、TB 级存储。不做集群——见《05-优化与踩坑记录.md》第 7 节关于集群的取舍说明。

2. 技术栈

层次	选型	版本	说明
操作系统	Debian	13.5 (trixie)	内核 6.12
运行时	OpenJDK (headless JRE)	21.0.12	LTS
后端框架	Spring Boot	4.1.0	Jakarta EE 10
后端基座	RuoYi-Vue-Plus	6.0.0	用户/角色/部门/菜单/字典等后台骨架 (MIT 许可)
数据库	PostgreSQL	17.11	主库
缓存/会话	Redis	8.0.2	Sa-Token 会话、限流、防重复提交
对象存储	MinIO	RELEASE.2025-04-22	文档原件、预览文件、缩略图
Web 服务器	nginx	1.26.3	静态资源 + 反向代理 (唯一对外入口)
前端	Vue 3 + Vite 5 + Element Plus 2.14 + Pinia	—	自研界面, 非 RuoYi 前端
文档预览	OnlyOffice Document Server (可选)	23.7 (docker 镜像)	Office 在线预览/编辑
文档转换	LibreOffice	25.2.3	Office → PDF 预览; jodconverter 调用
视频处理	ffmpeg	7.1.5	取帧做缩略图、转 mp4 预览
检索	PostgreSQL 全文检索	—	`tsvector` + GIN 索引 (中文用 `simple` 配置, 不装分词扩展)

2.1 后端模块结构

后端是 RuoYi-Vue-Plus 的多模块 Maven 工程，**业务代码集中在 `ruoyi-modules/ruoyi-doc`**:

...

dms-app/

└─ ruoyi-admin/ 启动模块（打包成可执行 jar）

└─ ruoyi-common/ 框架通用能力（mybatis/redis/security/log/oss/mail/doc...）

└─ ruoyi-modules/

| └─ ruoyi-system/ 用户、角色、菜单、部门、字典、参数、审计骨架

| └─ ruoyi-doc/ ★ DMS 业务模块（本次开发的全部内容）

| └─ controller/ 18 个 REST 控制器

| └─ service/impl/ 业务实现

| └─ mapper/ MyBatis-Plus Mapper

| └─ domain/ dto/ vo/ 实体与传输对象

| └─ enums/ PermissionFlag、AuditAction 等

| └─ job/ 定时任务（tus 清理、回收站清理、打包任务恢复/清理）

| └─ resources/db/migration/ 版本化 SQL 迁移

└─ ruoyi-extend/ SnailJob 调度、监控（本系统未启用，保留框架能力）

...

2.2 前端结构

...

dms-app-frontend/

└─ src/api/ 接口封装（http.ts 统一处理 RuoYi 的 JSON 信封与 401）

└─ src/stores/ Pinia: user（登录态/权限串）、site（站点配置）、clipboard（复制剪切）

└─ src/router/ 路由 + 权限守卫（按权限串/超管专属拦截）

└─ src/types/doc.ts 权限位、档位、文件分类的**唯一**定义处

- └─ src/views/doc/ 文档浏览器（列表/网格/大图三种视图、预览、权限、移动、回收站…）
- └─ src/views/system/ 用户、角色、部门、系统参数、审计日志、系统信息、站点配置
- └─ src/views/auth/ 登录、注册、忘记密码
- ...
-

3. 关键技术设计

3.1 权限模型（本项目最核心的设计）

****四个档位 + 一个独立开关****，底层是位掩码（`PermissionFlag`）：

档位	位掩码	组成	说明
禁止访问	256	拒绝位	命中即完全不可见（列表与搜索都不出现），向下继承，优先级最高
只读	3	可见 + 预览	
读写	55	只读 + 编辑 + 删除 + 上传	不含下载
完全控制	191	读写 + 下载 + **授权**	可以在该资源上给别人授权
下载	8	独立开关	可叠加在只读/读写之上

设计要点：

1. ****档位必须嵌套****（只读 ⊂ 读写 ⊂ 完全控制）。因为多主体授权取并集，只要嵌套成立，「禁止访问 > 读写 > 只读」这个优先级就自动满足，不需要额外写优先级比较逻辑；

唯一需要显式处理的是「拒绝位压过一切」。

2. ****下载是独立位****，不放进档位里。现场的真实诉求是「能看不能带走」，所以只读/读写默认不含下载，

需要时单独叠加。

3. ****授权能力有两道闸门****（缺一不可）：

- 能力闸门：权限串`doc:perm:grant`（或`doc:perm:full`），由超级管理员在「角色管理」里勾给某个角色；
 - 范围闸门：在该目录/文件上持有「完全控制」位。
- 只留能力闸门 → 被授权者能改任意目录的授权（越权）；只留范围闸门 → 拿到完全控制的人可以给自己复制权限（自我提权）。两道都在，才能既让多人分担维护、又不至于权限滚雪球。
4. 授权的三个主体：****用户 / 角色 / 部门****。取并集；`DENY`一票否决；目录授权可向下继承（`inherit\_to\_children`）；
- 支持`expires\_at`到期自动失效。

3.2 谁是真超管：按用户 ID，不按角色

系统里只有一个****真正的****超级管理员：`SystemConstants.SUPER\_ADMIN\_USER\_ID`（雪花 ID，写死在常量里）。他的权限不是靠角色，而是`SysPermissionServiceImpl`按用户 ID 注入

`\*:\*:\*`通配权限串，Sa-Token 把`\*:\*:\*`视为通配，所有`@SaCheckPermission`直接通过。

为什么这样设计：****角色数据是可以被改的，用户 ID 不可以****。如果超管资格来自某个角色，那么一旦有人误删了角色-菜单关联、或把该角色错配给别人，就可能出现"把自己锁在系统外"的情况。

按 ID 判定意味着：数据库里的授权数据全乱了，这个账号依然能登进来救场。

「超级管理员」那个****角色行是空壳****（`sys\_role\_menu`里 0 条记录），只作为显示存在，不允许分配给其他用户。给谁分配了都不会产生任何权限，反而会在用户列表里显示成"这人也是超管"，

把「谁是管理员」这件事搞糊涂。

各能力的边界（代码位置见括号）：

只有内置超管能做（兜底层）	可由超管勾给其他角色（能力串）
角色权限分配 （`SysRoleController.editPermission`）	审计日志查看`system:audit:list`
菜单管理写接口（上游	审计日志导出

`@SaCheckRole(superadmin)`)	`system:audit:export`
清除审计日志 (`AuditController.clear`)	系统信息 `system:info:list`
站点配置 (`SiteController`)	文档授权 `doc:perm:grant` / `doc:perm:full`
新建/调整顶层文档区 (`FolderController`)	
操作内置超管账号 (框架 `checkUserAllowed`)	

3.3 上传：tus 断点续传

现场网络不稳、文件大（单文件上限 10GB），所以用 tus 协议而不是普通表单上传：

- 前端 `tus-js-client` + `@uppy/tus`；后端 `tus-java-server`，落盘在
`/var/dms/tus`；
- 上传时以 `Upload-Metadata` 携带 `filename` 与 `folderId`（Base64），完成时由

`/api/upload/{id}/complete` 触发入库、生成缩略图/预览；

- ****服务端必须有人定期清理临时文件****：tus 自带的过期机制不会自己跑，中断的上传会一直留在磁盘上。

`TusCleanupJob` 每天 04:20 先调 tus 官方 `cleanup()`，再按目录时间戳兜底删掉超过 48 小时未变动的残留，

占用超过 5GB 打 ERROR 日志作为告警触发点。

3.4 预览：三条路径

文件类型	预览方式
图片	原图（或缩略图）+ PhotoSwipe 全屏图集
视频	服务端转 mp4（浏览器兼容性）+ Plyr 播放器；取帧做缩略图
音频	浏览器原生播放（Plyr）
PDF / Office / CAD / 压缩包等 75+ 种	OnlyOffice 在线查看（iframe）
未装 OnlyOffice 时	PDF/Office → 服务端 LibreOffice 转 PDF → pdf.js

	渲染
--	----

****为什么 Office 要两条路****: OnlyOffice 提供原生观感与在线编辑，但它是 3.3GB 的独立服务；

LibreOffice 转换则在应用内完成，不依赖外部服务。两者并存让"要不要装 OnlyOffice"成为一个可选项，

而不是硬依赖（见《03-部署手册.md》第 7 节）。

****图片/视频为什么需要签名直链****: `` / `` 这类浏览器原生请求带不上

`Authorization` 头，直接指向 `/preview` 会被判未登录（后端返回 200 + `code:401`，浏览器只当"加载失败"）。所以后端签发一条短期 HMAC 令牌（`dms.media.token-secret`，

默认 120 分钟），前端先换令牌再引用；令牌路径 `/api/doc/media/\*\*` 在 `security.excludes` 白名单里，

但控制器每次访问都会****重新校验该用户对该文件的预览权限****。

3.5 打包下载：小包同步、大包异步

目录打包下载（zip）由 nginx 关闭缓冲、后端流式写出，但****大目录****如果同步打包，请求会长时间挂着甚至超时。所以：

- 小于阈值（默认 200 个文件 / 500MB）→ 同步流式打包；
- 超过阈值 → 转异步任务（`doc\_export\_task` 表 + 后台线程池），前端在「导出任务」页看进度；
- 异步任务有并发上限（每用户活跃任务数）、过期时间（保留 24 小时）、

以及"僵尸任务恢复"定时任务（进程重启后把卡在 RUNNING 的任务重新捡起来）。

3.6 审计：旁路记录，不影响业务

所有敏感操作经 `AuditService.record(...)` 写 `doc\_audit\_log`。三条设计约束：

1. ****旁路****：审计写入失败绝不能让业务失败（写日志本身是次要目的，业务成功是主要目的）；
2. ****窗口函数取文件名****：日志里要记"谁下载了什么"，所以同时存 `resource\_id` 与 `resource\_path`

（冗余存名字，事后改名也能追溯当时的名字）；

3. ****不可篡改****：只提供查询与导出，界面没有修改入口；清除日志是不可逆操作且会销毁追责证据，

因此**只有内置超管**能做，且清除后立刻补记一条`AUDIT\_CLEAR`（如果先记再删，这条自己也会被删掉，

就成了"日志被清过但查不到是谁清的"）。

3.7 站点配置：可改的站名与标识图

站点名称、标识图、标签页图标、备案/版权、注册开关、SMTP 参数都存在`sys\_site\_config`（单行表），

公开读接口`/api/site/config`（登录页未登录时就要显示），写接口限超管。

- **标识图与 favicon 是两个文件**：favicon 是 16×16 的标签页小图，标识图是登录页标题上方、

登录后页头左上角那张给人看的图（建议 128×128 以上）。都存`/opt/dms/site-assets/`。

- 前端登录页按 64px、页头按 26px 等比缩放（`object-fit: contain`），所以无论上传多大都不会撑破布局。
- 缓存：URL 上带`?v=<更新时间戳>`，换图后立即生效，不用清浏览器缓存。

3.8 系统信息页：观测优先

「系统信息」页（`/system/info`，权限串`system:info:list`）把运维第一眼要看的东西集中在一屏：

业务数据量 → 服务器 → 应用（JVM）→ 磁盘 → 数据库 → MinIO → Redis。

两条硬规矩：

1. **采集失败不能让页面挂掉**：MinIO 挂了、Redis 连不上时，其余部分照常显示，对应分组里给一个`error`字段说明原因（运维页面最忌讳"整页白"）；
2. **不做重活**：对象统计有上限（20 万），避免桶里有几十万个对象时把采集请求拖死。

安全上有一条红线：**MinIO 的 Secret Key 绝不出现在接口返回里**。页面上只显示 Access Key 的掩码

（保留前 4 后 4）与"Secret Key 已配置"状态。这条有自动化断言守护：

`tools/verify\_system\_info\_page.py` 会拿服务器上真实的 secret 去返回体里搜。

4. 数据模型

4.1 业务表（`ruoyi-doc` 模块，前缀 `doc\_`）

表	作用	关键点
`doc_folder`	目录树	`parent_id` 建树，`folder_path` 物化路径便于子树查询；软删 `deleted_at`
`doc_file`	文件	`storage_key` 指向 MinIO 对象；`preview_key` / `thumbnail_key` 为转换产物；`search_vector` 全文检索列
`doc_file_text`	文件正文文本	供全文检索，与 `doc_file` 级联删除
`doc_folder_permission`	目录授权	唯一键 (folder_id, subject_type, subject_id)
`doc_file_permission`	文件授权	同上，粒度到单文件
`doc_audit_log`	审计日志	只增不改
`doc_export_task`	异步打包任务	进度、过期时间、归属用户
`doc_upload_session`	上传会话	tus 续传与去重（按文件 hash）
`sys_site_config`	站点配置	单行表 (id=1)

4.2 权限判定流程（`PermissionServiceImpl`）

...

computeUserFlags(resourceType, resourceId, userId)

- 超管 → 直接 FULL (191)
- 资源所有者（文档区 owner_id = userId）→ FULL
- 取当前用户的角色集、部门集

4. 沿目录链（自身 → 各级父目录）逐层收集 user/role/dept 三类授权，取并集

- 子目录继承父目录的授权（受 inherit_to_children 控制）

- 过滤已过期（expires_at < now）的授权

5. 任一命中 DENY(256) → 直接返回 0（拒绝位一票否决，且向下继承）

6. 文件的创建者额外获得 DELETE 位（自己传的文件可以删）

7. 去掉 DENY 位后返回

...

****性能****：目录链只在必要时回溯，批量场景（文件列表）用 `computeFileFlagsBatch` 按目录缓存

目录链结果——同一目录下的 50 个文件只算一次祖先链，避免 $N \times \text{深度}$ 次查询。

4.3 雪花 ID

所有主键是雪花 ID（19 位、约 $1.76e18$ ），**超出 JavaScript 安全整数范围 (2^{53})**。

因此：

- 后端把所有 `Long` 统一序列化成字符串（框架内置 `BigNumberSerializer`）；
- 前端任何地方都不允许 `Number(id)` 或 `parseInt(id)` ——一旦转换就静默丢精度，

表现为“授权给了 A，实际授给了 B”这种极难排查的错。前端 `types/doc.ts` 与各弹窗都按字符串处理。

5. 关键流程时序

5.1 上传并预览

...

浏览器 —— tus 分片上传 ——► `/api/upload/tus`（落盘 `/var/dms/tus`）

| 完成后

▼

POST `/api/upload/{id}/complete`

└─ Tika 识别 MIME

└─ 写 doc_file (storage_key 指向 MinIO files/...)

└─ Office → LibreOffice 转 PDF → previews/...

└─ 视频 → ffmpeg 转 mp4 + 取帧 → thumbs/...

└─ 抽正文文本 → doc_file_text (供全文检索)

└─ 审计 UPLOAD

用户点开文件 —► GET /api/doc/files/{id}/media (换 HMAC 短期直链)

└─ 图片/视频: / 直接引用签名地址

└─ Office/PDF: GET /api/onlyoffice/config 拿编辑器配置 → iframe 加载 OnlyOffice

...

5.2 授权

...

超管在「角色管理 → 分配权限」勾选 doc:perm:grant 给某角色

↓ (能力闸门)

被授权人拥有「完全控制」的目录上右键 → 权限设置

└─ 目标: 用户 / 角色 / 部门

└─ 档位: 只读 / 读写 / 完全控制 / 禁止访问 (+ 下载开关)

└─ 是否向下继承 / 到期时间

└─ 后端校验: 能力串 + 目标资源 FULL_CONTROL (范围闸门)

└─ 审计 PERM_GRANT (记录被授权主体与权限文案)

...

5.3 打包下载

...

选中多项 → 下载所选 zip

└─ 后端预估体积/数量

└─ 小于阈值: nginx 关缓冲 → 后端边读 MinIO 边写 zip 流

└─ 超过阈值：建 doc_export_task 记录 → 后台线程池打包 → 写 /opt/dms/export

前端「导出任务」页轮询进度 → 完成后点下载

两种路径都记审计（打包下载记 DOWNLOAD + 明细）

...

6. 目录与端口约定

6.1 服务器目录

路径	内容	属主/权限
`/opt/dms/dms-app.jar`	应用	root:root 644
`/opt/dms/dist/`	前端静态资源	root:root, a+rX
`/opt/dms/config/application-prod.yml`	外部配置（含口令）	root:dms 640
`/opt/dms/logs/`	应用日志	dms:dms
`/opt/dms/export/`	异步打包产物	dms:dms
`/opt/dms/site-assets/`	站点标识图 / favicon	dms:dms
`/opt/dms/zip-async/`	打包临时目录	dms:dms
`/opt/dms/backup/`	备份集与脚本	root
`/opt/dms/sql/`	随包 SQL（供排障与升级）	root
`/var/dms/tus/`	tus 上传临时文件	dms:dms
`/var/dms/tmp/`	multipart 临时目录	dms:dms
`/opt/minio/data/`	对象存储数据	minio-user
`/root/.dms-credentials`	部署凭据（600）	root
`/root/.minio-credentials`	MinIO 凭据（600）	root

--	--	--

6.2 端口

端口	服务	监听	对外
22	sshd	0.0.0.0	仅内网（ufw 限制网段）
80	nginx	0.0.0.0	**唯一对外入口**
8080	DMS 应用	127.0.0.1	否
8081	OnlyOffice（可选）	127.0.0.1	否（经 nginx 同源反代）
9000/9001	MinIO API/控制台	127.0.0.1	否（控制台走 SSH 隧道）
5432	PostgreSQL	127.0.0.1	否
6379	Redis	127.0.0.1	否

7. 许可与来源声明

组件	许可	说明
DMS 业务代码（`ruoyi-doc` 模块、前端）	随本项目	自研
RuoYi-Vue-Plus 6.0.0	MIT	后端基座，本项目在其上二次开发
MinIO	AGPL-3.0	独立程序，离线包内为官方二进制，版本锁定见《02》
OnlyOffice Document Server	AGPL-3.0	独立服务，容器镜像
LibreOffice / PostgreSQL / Redis / nginx / ffmpeg	MPL-2.0 / PostgreSQL / BSD / BSD / LGPL/GPL	由 Debian 发行版提供

****注意****：MinIO 与 OnlyOffice 是 AGPL 组件，作为独立程序随发布包分发时，

接收方若再对外分发需自行履行 AGPL 的源码提供义务（本仓库不包含这两个组件的源码，只提供官方下载地址与镜像包）。